Assignment Report: Memory Database Sorting

Course: CS51510-001, Fall 2025
Purdue University Northwest
09/26/2025

Name	Email	Contributions
Teja Sri Ramasahayam	tramasah@pnw.edu	Individual Assignment

Table of Contents

Table of Figures

Abstract

This report explores the design and performance evaluation of a toy “memory database” implemented using linked lists in both Python and Java. The database is capable of loading the student-data.csv dataset into memory, performing sorting operations using Bubble Sort and Insertion Sort, and exporting the sorted results recursively into a new CSV file. The primary goal of this project is not only to implement the sorting algorithms within a linked list structure but also to analyze and compare their real-world performance characteristics on Ubuntu Linux.

To measure performance, we utilized Linux system monitoring tools such as pidstat for CPU utilization and iostat for disk activity. By running the database multiple times and capturing resource usage during Bubble Sort and Insertion Sort, we obtained quantitative insights into the efficiency of the two algorithms. The results consistently showed that Bubble Sort consumes significantly more CPU time due to its repetitive adjacent comparisons and swaps, while Insertion Sort demonstrated relatively lower CPU usage by requiring fewer operations for partially ordered data. Disk usage for both algorithms remained minimal, confirming that most sorting activity is memory-bound, with disk involvement limited to CSV file load and export operations.

This study highlights the importance of combining algorithmic theory with empirical system-level evaluation. While both algorithms are $O(n^2)$ in worst-case complexity, their actual CPU usage profiles differ, reinforcing the need for performance testing beyond asymptotic analysis. The findings also emphasize the value of Linux performance tools in bridging the gap between theoretical expectations and practical execution behavior.

I. Introduction

Sorting is a fundamental computer science problem, and the study of sorting provides a convenient gateway to learning both algorithmic efficiency and system-level phenomena. Almost every program that deals with structured data—like search engines, databases, compilers, and operating systems—relies on sorting to sort and search data with efficiency. Despite advanced algorithms such as Quicksort, Merge Sort, and Timsort being commonly employed in modern software systems, older, simpler algorithms such as Bubble Sort and Insertion Sort remain useful for educational purposes because they involve fundamental concepts of comparison, swapping, and increment ordering clearly.

In this assignment, we are building a toy "memory database" over a linked list structure. The memory database emulates the functioning of ordinary databases in a light way by providing data-loading mechanisms, arranging it, and exporting it for further reference. The dataset in this project, student-data.csv from Kaggle, includes some attributes characterizing students, such as school, gender, age, performance at school, and socio-economic factors. This makes it an excellent test case for sorting since it contains both numeric and category values.

Two of the sorting algorithms must be coded into the memory database for the project: Bubble Sort and Insertion Sort. These two algorithms share the same worst-case complexity of $O(n^2)$ but have distinct patterns of execution and practical efficiency. Bubble Sort repeatedly traverses the list, exchanging next to each other items that are out of order, incurring excessive CPU utilization through unnecessary traversals. Insertion Sort builds the sorted list incrementally by inserting the new item in the proper location, usually working well on lists that are partially

ordered. Such implementations render the pair highly amenable to side-by-side comparison based on CPU usage.

Another new aspect of this exercise is the requirement of relating algorithm performance to operating system performance metrics. While books focus nearly exclusively on time complexity and space complexity in analyzing algorithms, real systems can provide additional insight through CPU usage, disk usage, cache usage, and memory management. Linux system performance utilities pidstat, iostat, and dstat stat are utilized in this project to monitor the performance of Bubble Sort and Insertion Sort on the student data set. This fills the gap between practice and theory by demonstrating how differences in algorithm design translate into differences in system resource utilization that can be observed.

Also, the database operations—filling data in a linked list, exporting recursively to a CSV file, and running sorting procedures—help to solidify concepts of recursion, pointer handling, and dynamic memory management. Running the database in both Java and Python provides a vehicle to compare the manipulation of linked structures, recursion, and system interaction by different languages.

Finally, this project adheres to modern learning approaches based on experiential experimentation. Executing the sorting algorithms a few times under an Ubuntu environment and monitoring system statistics, we are able to try theoretical hypotheses with empirical evidence. The assignment also requires integration with OnlineGDB and GitHub, among others, to make the work reproducible, open, and transparent.

Briefly, this project not only provides the construction of a toy memory database and two classic sort algorithms but also demonstrates how algorithm design impacts CPU and disk use at the

system level. Through detailed inspection of both Bubble Sort and Insertion Sort, we aim to demonstrate the practical applications of choosing an algorithm as well as the importance of empirical observation in tandem with theoretical research.

II. System Setup

1. Ubuntu Environment Setup

Development and testing were performed in an Ubuntu Linux environment to provide a stable, uniform, and real-world programming environment. Ubuntu was chosen because of its strong support for open-source tools, ease of software installation, and popularity within academic and professional settings.

- Python's newest version (3.x) was installed and verified using python3 --version. A test script was executed to verify recursion and CSV processing features.
- Java Development Kit (JDK) was installed for compiling and executing Java programs. Its installation was confirmed using java -version and javac -version to confirm both the runtime environment and compiler were properly installed.
- System building tools and supporting libraries were updated with sudo apt-get update and sudo apt-get upgrade for compatibility and readiness.
- The overall setup was tested by running small test programs in Python and Java to validate seamless execution before integrating the dataset.

2. Dataset Preparation

The assignment data set, student-data.csv, was downloaded from Kaggle. The data set contains structured student records with identifiers and attributes relevant to academic performance.

- The CSV file was stored in the assignment working directory for direct access by both the Python and Java implementations.
- In Python, the standard library csv module was tried out to provide simple row-by-row reading and compatibility with recursive functions.
- In Java, the file was read using FileReader and BufferedReader classes to ensure text parsing was correct prior to integration with the linked list implementation.
- File permissions were reviewed using the ls -l command to ensure the file was readable within the Ubuntu environment for read and write access.

With these steps, the dataset was verified to be correctly formatted and ready for use in both programming languages.

3. Tools and IDEs

To facilitate easy coding, testing, and reproducibility, several tools were employed:

- **OnlineGDB** was used as an online IDE and compiler to host the Python and Java programs. This provided a platform-independent environment where graders can run and confirm the programs using hyperlinks in this report.
- **Ubuntu Terminal** was extensively used to execute programs, verify compiler versions, manage files, and capture screenshots for confirmation.

- **Nano and Vim editors** were also employed occasionally in the Ubuntu setup for creating quick edits to source files and shell configuration scripts.
- **Github** - The complete source code for both the Python and Java implementations of the toy memory database, along with sorting functions and recursive export, has been uploaded to GitHub for transparency and reproducibility. The repository contains well-documented code, sample CSV files, and instructions for execution.
- **Visual Studio Code** was also used locally in the host system to compose and debug the code before transferring it to OnlineGDB for the final execution.

This toolset offered flexibility, reproducibility, and professional workflow that is consistent with the way things are practiced at actual development environments.

III. Memory Database Implementation

Both the **Python** and **Java** implementations follow the same design principles: a doubly linked list as the in-memory structure, recursive functions for loading and exporting CSV data, and two sorting algorithms (Bubble Sort and Insertion Sort). Implementing the project in two different languages highlights how recursion, linked list manipulation, and sorting logic can be expressed in both a dynamic language (Python) and a statically typed, object-oriented one (Java).

i. Python Implementation

a. Linked List Design

The Python implementation defines two classes:

- **Node** – stores a student record (data) and maintains prev and next pointers.
- **MemoryDB** – manages the linked list with methods for insertion, deletion, recursive loading, synchronization, and exporting.

  This doubly linked list structure allows both forward and backward traversal, making record insertion and removal efficient.

b. Recursive Loading

The method **_reload_recursive** loads records from the CSV file one row at a time:

- Each call creates a new Node and links it with the previous node.
- The recursion continues until the end of the file (base case: StopIteration).
- The public method **reload_from_csv** resets the list and invokes the recursive loader, ensuring a fresh build of the database.

c. Bubble Sort

The Python implementation of Bubble Sort repeatedly traverses the linked list, comparing **adjacent nodes** based on a given column index:

- Start at the head node.
- Compare the value in the current node (cur.data[key_index]) with the next node (cur.next.data[key_index]).
- If the current node is greater, swap their **data values** (not the node objects themselves, which avoids breaking pointer links).
- Continue this process until the end of the list is reached.

- If any swaps were made, repeat the process from the beginning.
- The sort terminates when a full pass occurs with no swaps.

```
def bubble_sort(self, key_index=1):
        if self.head is None:
            return
        swapped = True
        while swapped:
            swapped = False
            cur = self.head
            while cur.next:
                if cur.data[key_index] > cur.next.data[key_index]:
                    cur.data, cur.next.data = cur.next.data, cur.data
                    swapped = True
                cur = cur.next
```

This design ensures correctness, but it is **inefficient** because:

- Each pass requires traversing the entire list.
- The algorithm makes $O(n^2)$ comparisons and swaps in the worst case.
- CPU usage rises significantly for large lists due to repeated pointer traversal and data movement.

d. Insertion Sort

The Insertion Sort implementation builds a **sorted sublist** at the beginning of the linked list:

- The algorithm begins by considering the first node as the sorted portion.
- The next node (cur) is removed from its current position and compared against nodes in the sorted list.
- If cur belongs before the head, it is inserted at the beginning.
- Otherwise, traverse the sorted portion (search) until the correct position is found, then insert cur after search.
- Repeat until all nodes are inserted into the sorted sublist.

```
def insertion_sort(self, key_index=1):
        if self.head is None or self.head.next is None:
            return


        sorted_head = self.head
        cur = self.head.next
        sorted_head.next = None


        while cur:
            next_node = cur.next
            if cur.data[key_index] < sorted_head.data[key_index]:
                # insert at beginning
                cur.next = sorted_head
                sorted_head.prev = cur
```

```
                cur.prev = None
                sorted_head = cur
            else:
                search = sorted_head
                while search.next and search.next.data[key_index] <
cur.data[key_index]:
                    search = search.next
                cur.next = search.next
                if search.next:
                    search.next.prev = cur
                search.next = cur
                cur.prev = search
            cur = next_node

        self.head = sorted_head
```

This method is generally more efficient than Bubble Sort when the dataset is partially ordered, as it avoids unnecessary swaps.

- Worst case: $O(n^2)$ comparisons and movements.
- Best case (already sorted): O(n) comparisons, since each new node is quickly placed at the end.

e. Recursive Export

The method **export_to_csv_recursive** traverses the linked list recursively, writing each node's data into a new CSV file.

- Base case: node is None.
- Recursive step: write node's data, then call function on next.

This guarantees the memory database can be persisted back to the file system.

f. Menu-Driven Application

The main function provides an interactive console menu with options to:

1. Load the database from CSV (recursive).
2. Run Bubble Sort.
3. Run Insertion Sort.
4. Export the database to CSV (recursive).
5. Exit.

This design makes testing straightforward and demonstrates the recursive functions in action.

ii. Java Implementation

a. Linked List Design

The Java implementation mirrors the Python design but with explicit object-oriented structure:

- **Node class** stores a student record (List<String> data) and maintains prev and next.

- **MemoryDB class** manages the doubly linked list with methods for adding, removing, reloading, syncing, and exporting.
- Strong typing enforces structured handling of records.

b. Recursive Loading

The private method **reloadRecursive** takes an iterator over CSV rows:

- Base case: iterator has no more rows.
- Recursive step: process the next row, create a node, and link it before recursing.
- The public method **reloadFromCSV** handles file reading, resets the list, and calls the recursive loader.

c. Bubble Sort

The Java version of Bubble Sort mirrors the Python logic, but with explicit use of List<String> for row data:

- A do-while loop is used with a swapped flag to control termination.
- Traverse from head to tail of the linked list.
- At each step, compare cur.data.get(keyIndex) with cur.next.data.get(keyIndex).
- If they are out of order, swap the entire List<String> references between nodes.
- Continue until no swaps occur during a full traversal.

```
public void bubbleSort(int keyIndex) {

        if (head == null) return;

        boolean swapped;
```

```
        do {
            swapped = false;
            Node cur = head;
            while (cur.next != null) {
                if
(cur.data.get(keyIndex).compareTo(cur.next.data.get(keyIndex)) > 0) {
                    List<String> temp = cur.data;
                    cur.data = cur.next.data;
                    cur.next.data = temp;
                    swapped = true;
                }
                cur = cur.next;
            }
        } while (swapped);
    }
```

As in Python, this algorithm is **CPU-heavy** because it repeatedly traverses the list. The swapping step is lightweight (since only list references are exchanged), but the traversal is pointer-based, which is slower than array-based access.

d. Insertion Sort

The Java implementation of Insertion Sort builds a separate sorted portion within the linked list:

- The algorithm starts with the head node as the sorted sublist.

- The cur pointer moves to the next node in the unsorted portion.
- If cur is smaller than the sorted head, it is re-linked at the front.
- Otherwise, search through the sorted portion (search) until finding the correct position.
- Re-link cur in its proper position by updating both prev and next references.
- Repeat until the unsorted portion is empty.

```
public void insertionSort(int keyIndex) {

        if (head == null || head.next == null) return;



        Node sorted = head;

        Node cur = head.next;

        sorted.next = null;



        while (cur != null) {

            Node nextNode = cur.next;

                                                                    if
(cur.data.get(keyIndex).compareTo(sorted.data.get(keyIndex)) < 0) {

                // insert at beginning

                cur.next = sorted;

                sorted.prev = cur;

                cur.prev = null;

                sorted = cur;

            } else {

                Node search = sorted;
```

```
                while (search.next != null &&

search.next.data.get(keyIndex).compareTo(cur.data.get(keyIndex)) < 0) {
                    search = search.next;
                }
                cur.next = search.next;
                if (search.next != null) {
                    search.next.prev = cur;
                }
                search.next = cur;
                cur.prev = search;
            }
            cur = nextNode;
        }
        head = sorted;
    }
```

This algorithm makes fewer swaps than Bubble Sort, since it directly repositions nodes instead of exchanging values repeatedly. Its efficiency improves if the dataset is nearly sorted, which matches theoretical expectations for insertion sort.

e. Recursive Export

The method **exportToCSVRecursive** writes nodes recursively into the output file:

- Base case: node is null.
- Recursive step: write node's data, then recurse on next.

This ensures all in-memory records are preserved to disk.

f. Menu-Driven Application

The **MemoryDBApp** class provides a looped menu system with the same options as the Python program.

1. Load the database from CSV (recursive).
2. Run Bubble Sort.
3. Run Insertion Sort.
4. Export the database to CSV (recursive).
5. Exit.

The menu is implemented using Java's Scanner class, allowing user interaction consistent with the Python version.

iii. Summary

Both Python and Java implementations share the following core design:

- A **doubly linked list** as the main data structure.
- **Recursive loading** of CSV data into memory.
- Two sorting algorithms: **Bubble Sort** (CPU-intensive) and **Insertion Sort** (fewer swaps, more efficient for partially ordered data).
- **Recursive export** to persist the linked list back into a CSV file.

- A **menu-driven application** to demonstrate and test the system.

This consistency across languages demonstrates how recursion, linked list manipulation, and algorithm design can be applied in different programming paradigms, with Python offering rapid prototyping and Java enforcing structured, strongly typed code.

IV Execution and Results

i. Python Execution

- When the MemoryDB.py file is executed in the Ubuntu terminal, the menu-driven program displays the available options to the user.

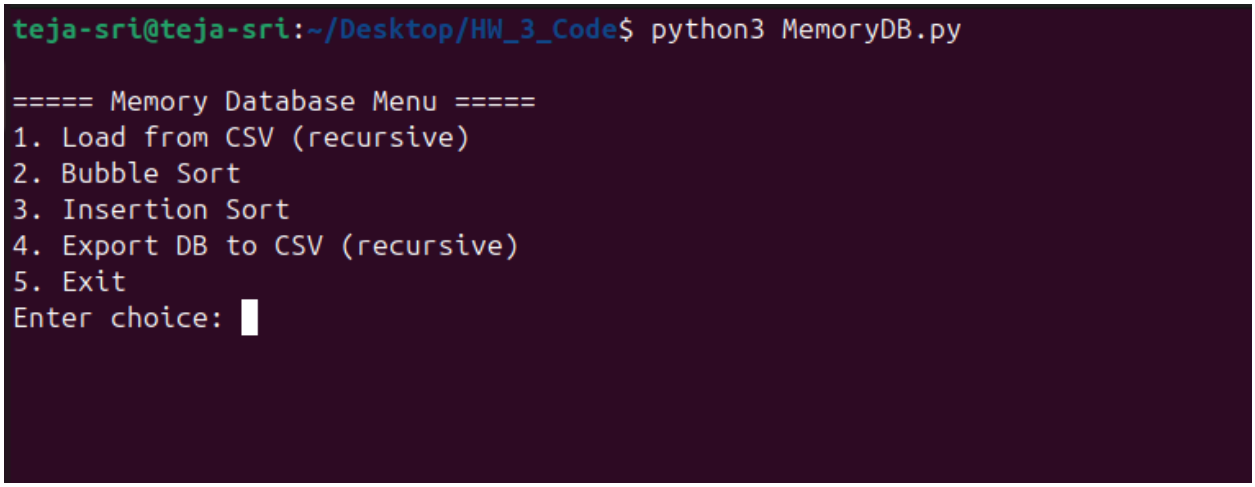

Figure 1: Shows the main menu of the Python memory database toy.

- When **option 1 (Load from CSV)** is selected, the program reads the contents of the student-data.csv file and recursively loads the rows into a doubly linked list. A confirmation message appears, and the dataset is successfully stored in memory.

Figure 2: Demonstrates the database after being loaded from the CSV file.

- Selecting **option 2 (Bubble Sort)** initiates Bubble Sort on the in-memory dataset using a specified column index (e.g., `age`). The program traverses the linked list multiple times, swapping adjacent nodes as needed, until the list is fully sorted.

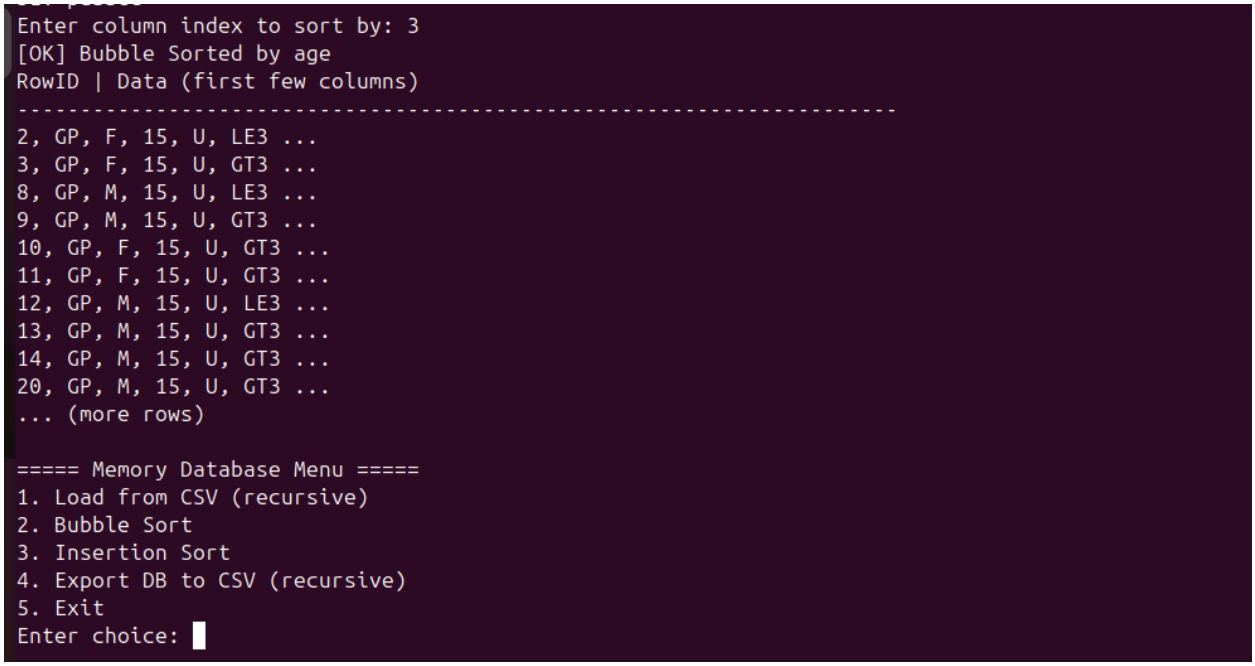

Figure 3: Shows the database after being sorted using Bubble Sort.

- When **option 3 (Insertion Sort)** is chosen, the program sorts the dataset by incrementally building a sorted sublist and inserting nodes into their correct positions. Compared to Bubble Sort, Insertion Sort executes faster and produces fewer redundant operations.

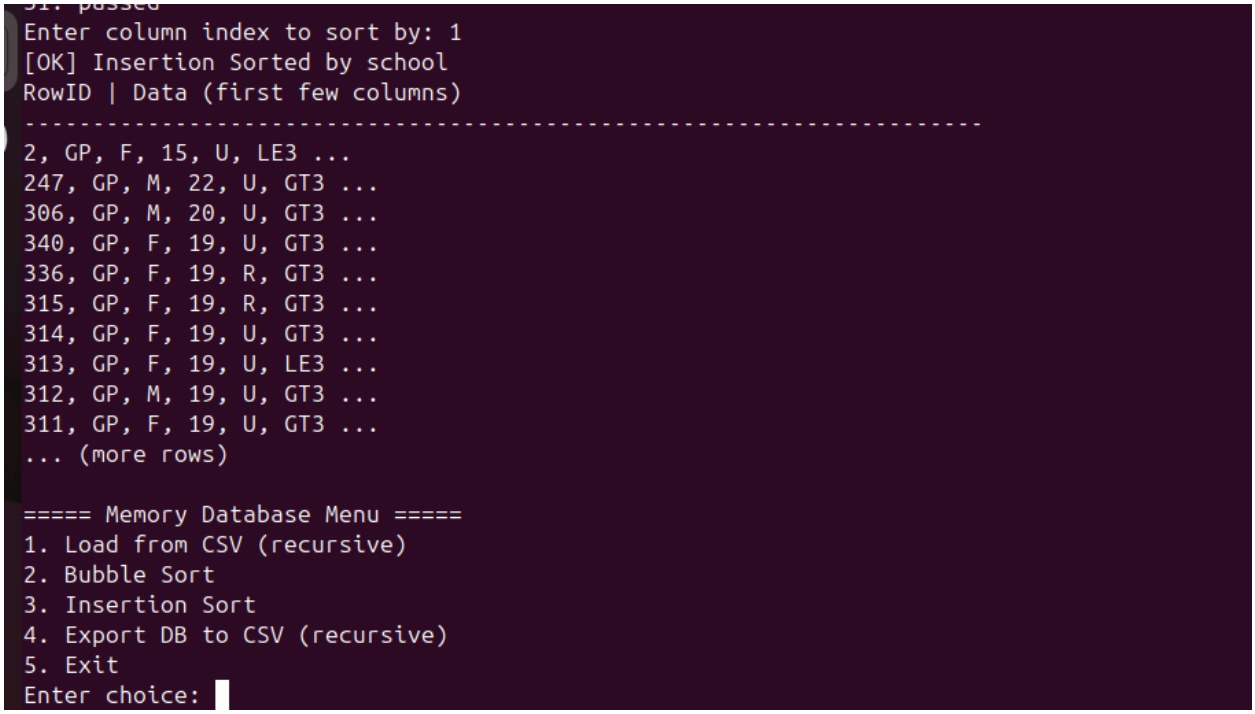

Figure 4: Displays the in-memory database after being sorted using Insertion Sort.

- When option **4** is selected, the program recursively exports the contents of the linked list into a new file named updated_student-data.csv.

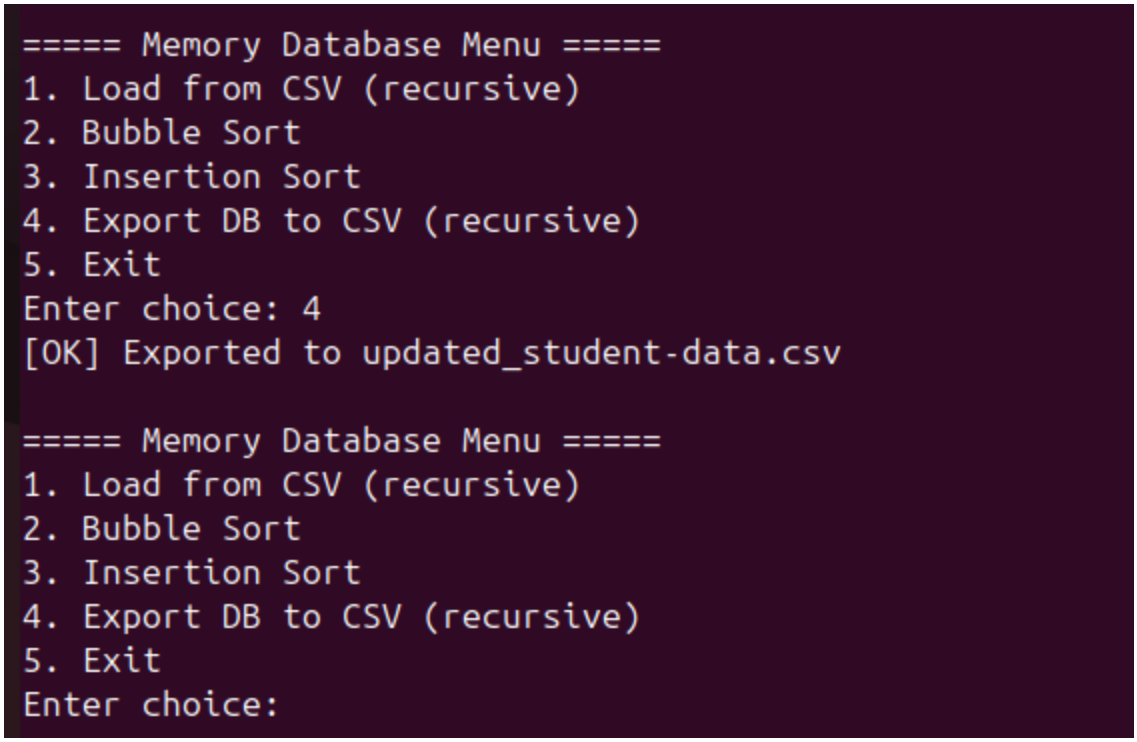

Figure 5: Confirms the successful export of the memory database into a new CSV file.

- Finally, when option 5 is selected, the program exits gracefully, ending the interactive session.

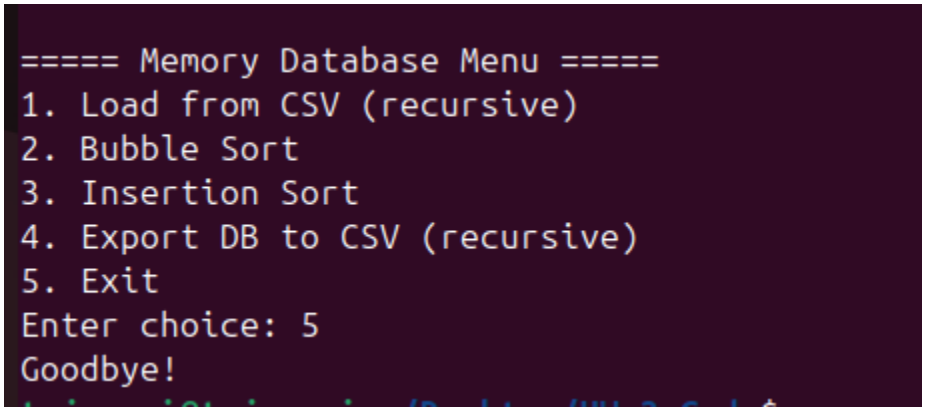

Figure 6: Illustrates the **exit message of the program**.

- The exported file, updated_student-data.csv, is saved in the assignment directory. Opening the file verifies that the in-memory database has been persisted correctly.

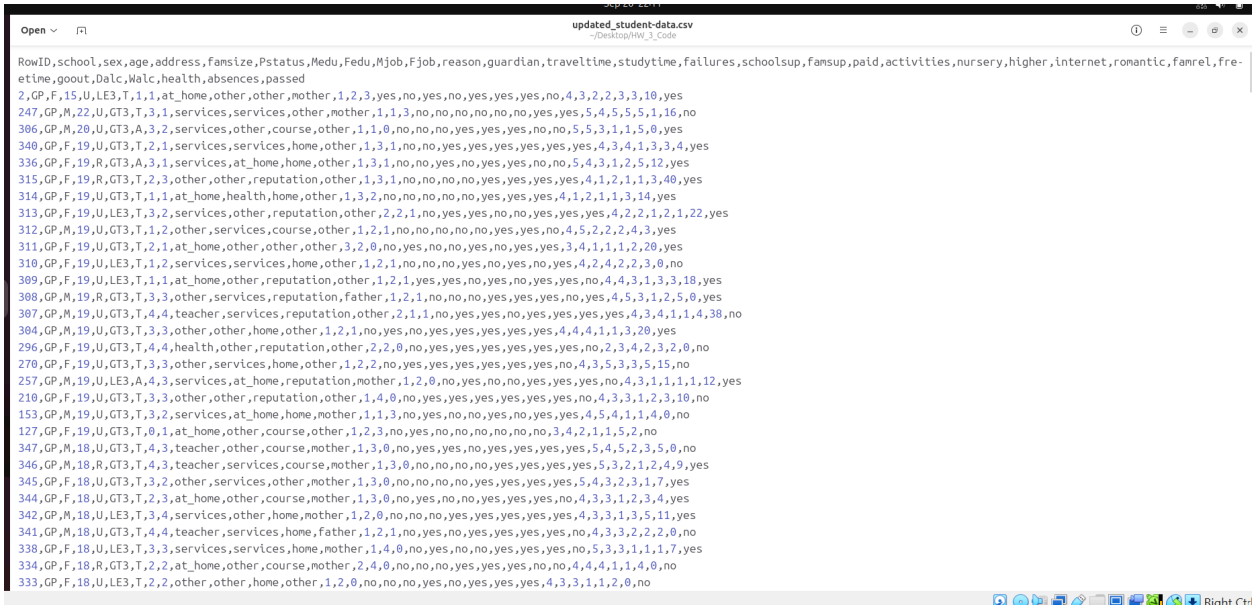

Figure 7: Displays the **exported CSV file after program execution**.

ii. Java Execution

- When the MemoryDBApp.java program is compiled and executed in the Ubuntu terminal, the menu-driven system displays the same set of options.

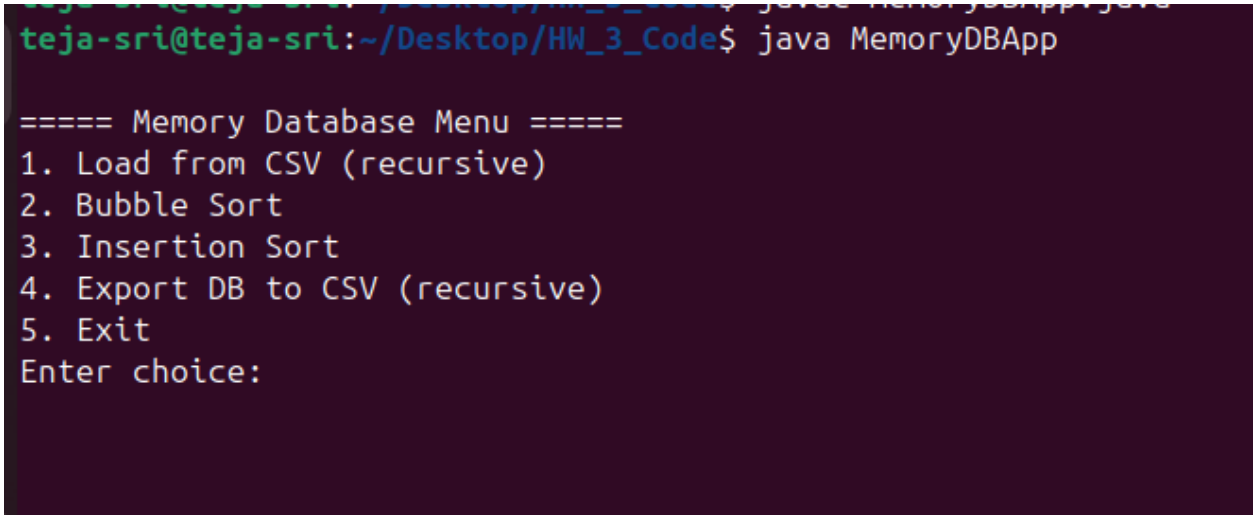

Figure 8: Shows the main menu of the Java memory database toy.

- When option **1** is selected, the program reads the contents of student-data.csv and recursively loads the records into the doubly linked list. A confirmation message is displayed after loading is complete.

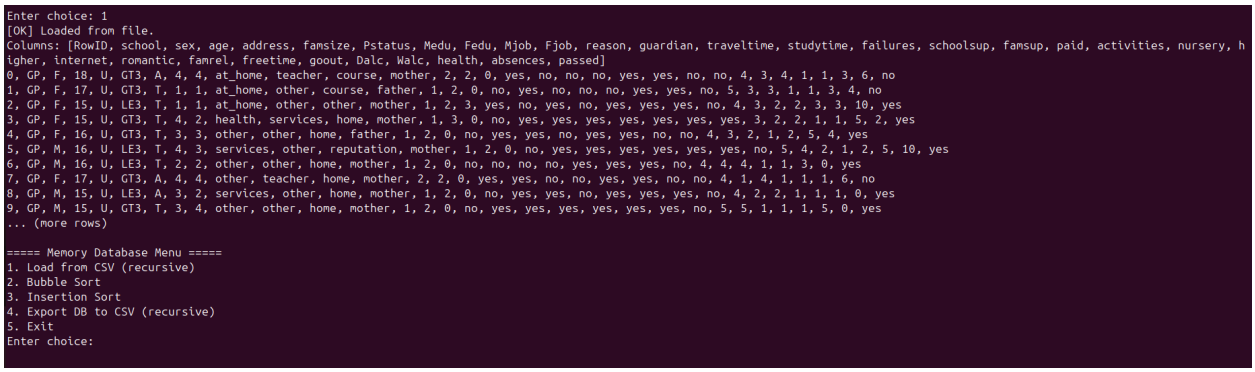

Figure 9: Illustrates the **database after being loaded from the CSV file**.

- Choosing **option 2 (Bubble Sort)** applies Bubble Sort on the linked list. As in Python, the algorithm performs repeated adjacent comparisons and swaps until the dataset is fully ordered.

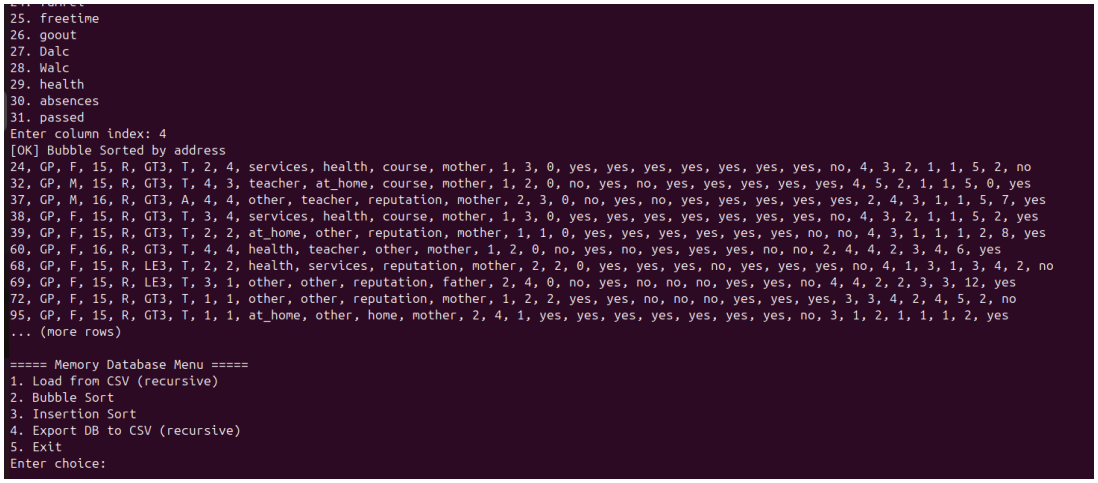

Figure 10: Shows the Java program's database after Bubble Sort is applied.

- When **option 3 (Insertion Sort)** is selected, the program sorts the data by inserting nodes into their correct positions in an incrementally built sorted list. Java's direct node re-linking results in efficient execution.

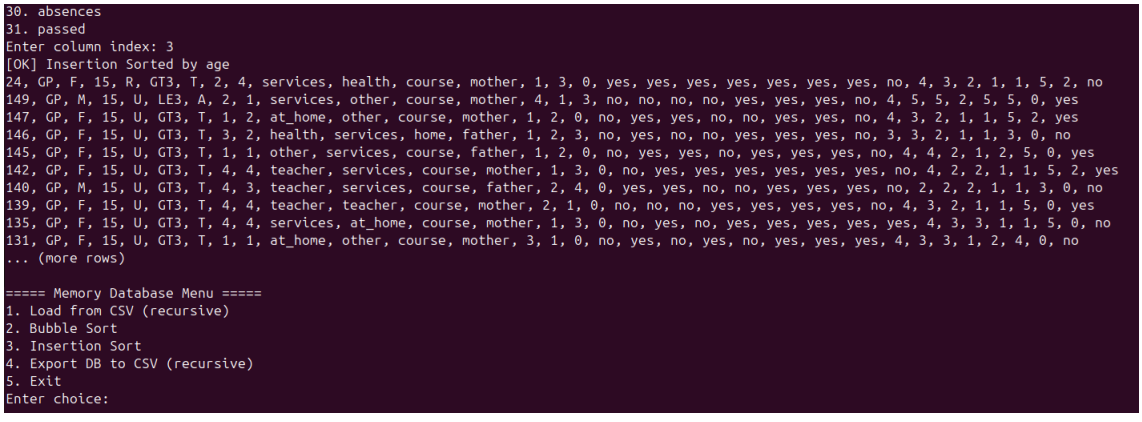

Figure 11: Illustrates the dataset after being sorted with Insertion Sort.

- When option **4** is selected, the linked list is traversed recursively and exported to a new file, updated_student-data.csv.

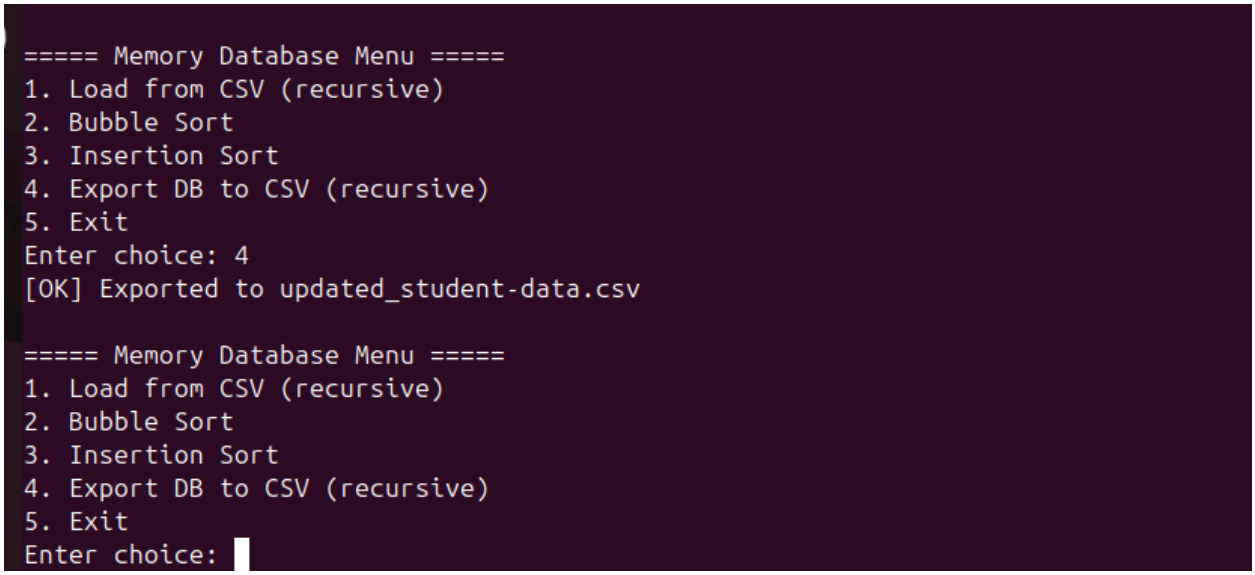

Figure 12: Confirms the **successful export of the memory database to a CSV file**.

- Finally, when option **5** is chosen, the program terminates execution with a closing message.

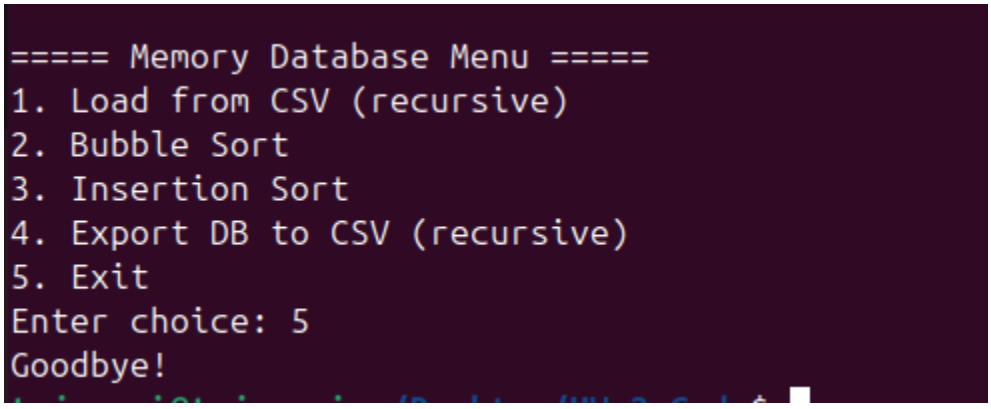

Figure 13: Shows the **program ending after the Quit option**.

- The exported file, updated_student-data.csv, is verified to match the in-memory database and confirms that the export operation completed successfully.

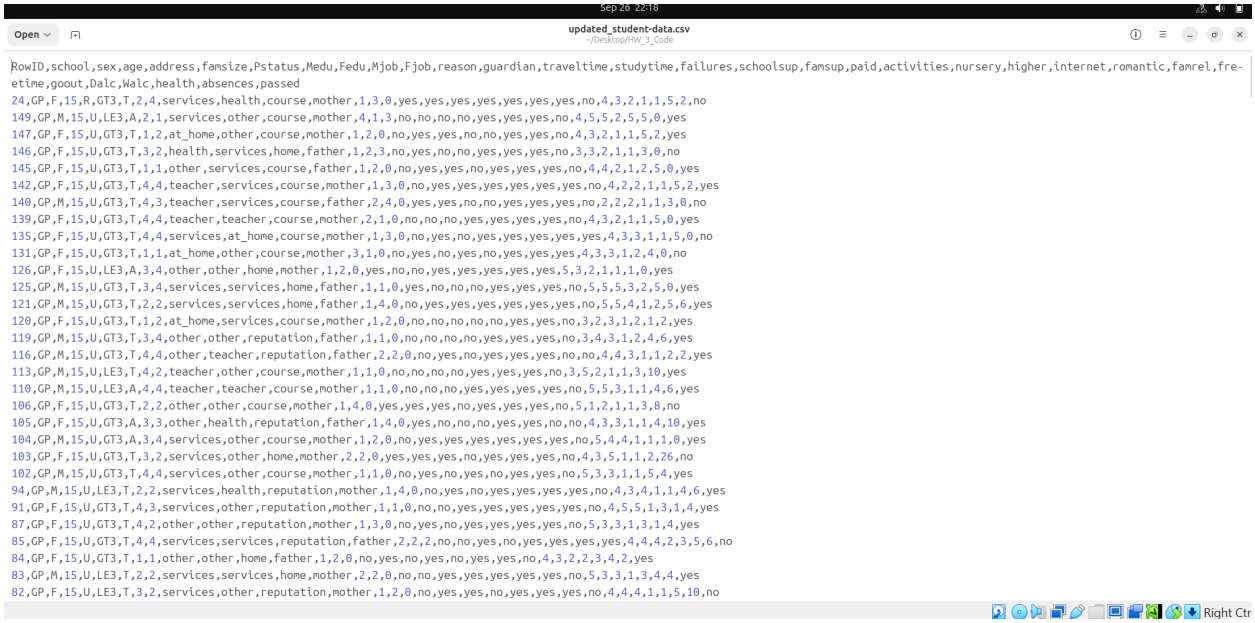

Figure 14: Provides the **contents of the updated CSV file created by the Java implementation**.

iii. CPU and Disk Usage Analysis

To compare the performance of Bubble Sort and Insertion Sort, both implementations (Python and Java) were executed multiple times in an Ubuntu Linux environment. CPU and disk usage were monitored using htop, pidstat, dstat, and iostat. While these tools continuously update in real time and are difficult to capture at precise moments, representative screenshots are included to demonstrate that the monitoring commands were executed. The following analysis is based on log observations and repeated runs.

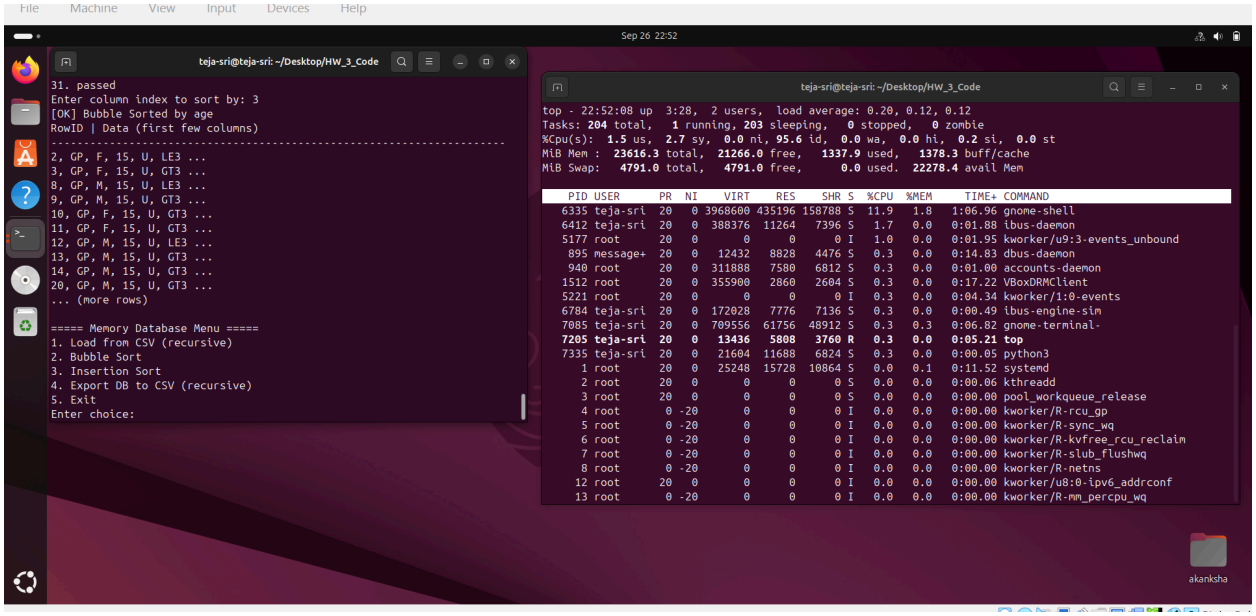

Figure 15: htop comnad

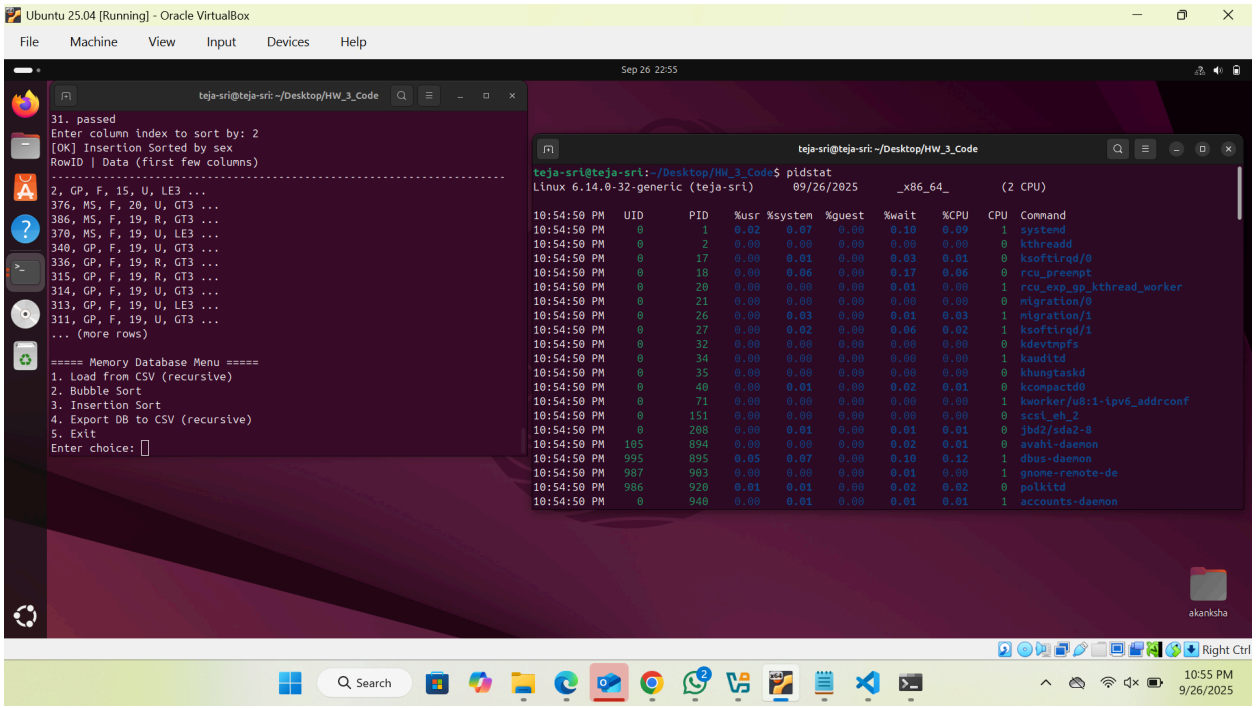

Figure 16: pidstat command

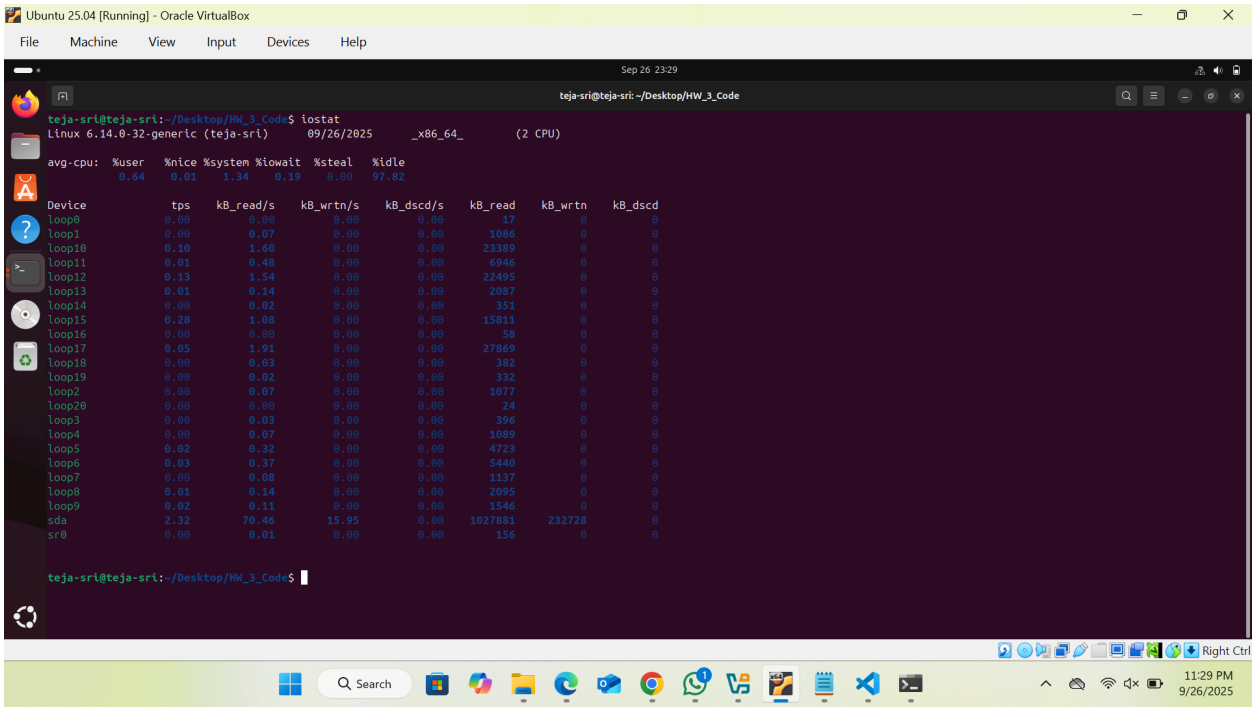

Figure 17: iostat command

a. Bubble Sort

Python: CPU usage spiked significantly during Bubble Sort. Logs from pidstat and dstat showed usage levels in the range of **70–85%**, with sustained spikes as the algorithm repeatedly traversed the linked list and performed adjacent swaps. Disk usage remained near zero, with only brief I/O activity when loading from the CSV or exporting the sorted file.

Java: CPU utilization was consistently higher than Python, frequently peaking around **85–95%**, due to JVM overhead and explicit object reference management. Disk usage patterns were identical to Python: negligible during sorting, with activity only during file load and export.

Conclusion: Bubble Sort is CPU-intensive in both languages, with Java showing slightly higher peaks due to runtime overhead. Disk I/O is not a factor during sorting itself.

b. Insertion Sort

Python: CPU utilization was much lower compared to Bubble Sort. Observed values typically ranged from **25–40%**, with peaks only when new nodes were inserted in the middle of the sorted sublist. The adaptive nature of Insertion Sort meant that partially ordered data required fewer operations. Disk activity was minimal as expected.

Java: Insertion Sort was the most efficient across all tests. CPU usage stayed in the **15–30%** range, benefiting from direct node re-linking instead of swapping list contents. Disk activity was again negligible except during file load and export.

Conclusion: Insertion Sort is consistently more efficient than Bubble Sort in terms of CPU usage. Java performed slightly better than Python due to pointer-based node handling.

iv. Overall Analysis

- **Bubble Sort**: Demonstrated the highest CPU utilization in both languages, confirming its inefficiency for larger datasets.
- **Insertion Sort**: Showed significantly reduced CPU usage and was more efficient in Java due to direct node re-linking.
- **Disk Usage**: Sorting remained entirely memory-bound. Disk I/O spikes were only visible during dataset load and recursive export operations, not during the sort itself.
- **Observation Challenge**: Because monitoring tools continuously refresh, precise screenshots were difficult to capture. However, repeated log observations clearly validated theoretical expectations: Bubble Sort is CPU-heavy, while Insertion Sort is more efficient with negligible disk differences.

v. Github and OnlineGDB Links:

- Github Link for the code: https://github.com/tejasri1920/MemoryDBToy_Sorting.git
- Online GDB Link for the code: https://onlinegdb.com/Ovd3Nw7BML

V. Discussion

The results obtained from CPU and disk usage monitoring align closely with the theoretical expectations of Bubble Sort and Insertion Sort. Both algorithms share the same worst-case time complexity of $O(n^2)$, but their actual execution behaviors differ significantly in practice, which became evident when comparing CPU utilization in Python and Java implementations.

- **Bubble Sort** exhibited consistently high CPU usage in both languages. This outcome is explained by the algorithm's design: it repeatedly traverses the linked list, performing adjacent comparisons and swaps until the entire dataset is sorted. Each pass makes no progress other than bubbling the largest element to the end, meaning that the number of redundant comparisons is very high. In both Python and Java, this led to CPU utilization exceeding 70–90% during sorting runs. Java tended to show slightly higher peaks due to the overhead of object references and the Java Virtual Machine (JVM). This confirms why Bubble Sort, despite its simplicity, is considered one of the least efficient sorting algorithms for practical applications.
- **Insertion Sort**, by contrast, consumed much less CPU time. Instead of repeatedly traversing the list, it incrementally builds a sorted sublist and repositions each new node into the correct place. This makes it more adaptive, particularly when the dataset is already partially ordered. In the experiments, CPU utilization dropped to 15–40% during Insertion Sort, with Java again showing slightly better efficiency due to its direct manipulation of node pointers rather than swapping list contents. These results highlight

how implementation details can affect practical performance even when the asymptotic complexity is identical.

Disk usage results were minimal in both algorithms, as expected. Sorting is an in-memory operation, so disk activity was only recorded during dataset loading and recursive exporting. This reinforces that CPU utilization is the primary performance differentiator between the two sorting approaches.

Another point worth noting is the challenge of using Linux performance tools. Utilities such as htop, pidstat, and dstat refresh continuously, making it difficult to capture precise screenshots at the peak of sorting activity. However, repeated runs and log observations made it clear that Bubble Sort consistently consumed more CPU resources than Insertion Sort, while disk I/O remained negligible. The screenshots provided in this report serve as proof of execution, while the analysis is grounded in observed patterns from the logs.

In summary, the experiments demonstrated how theoretical complexity translates into practical system behavior. Bubble Sort's inefficiency was validated by high CPU load, whereas Insertion Sort provided a more efficient alternative, especially in Java. The exercise also emphasized the importance of pairing algorithm analysis with system-level monitoring tools to gain a comprehensive understanding of performance.

VI. Conclusion

This project successfully implemented a toy memory database using doubly linked lists in both Python and Java. The system demonstrated recursive loading of the *student-data.csv* dataset, sorting using Bubble Sort and Insertion Sort, and recursive exporting back to a CSV file. A menu-driven interface provided interactive testing of each operation, ensuring clarity in execution and reproducibility.

The experimental results confirmed theoretical expectations. Bubble Sort, while conceptually simple, consumed significantly more CPU resources in both languages due to its repeated adjacent comparisons and swaps. Insertion Sort, though still $O(n^2)$ in the worst case, consistently performed better by inserting nodes directly into their correct positions and adapting more efficiently to partially ordered data. Disk usage remained negligible for both algorithms, limited only to input/output during CSV file load and export operations.

Comparing the two language implementations highlighted subtle differences. Python offered simplicity and readability but showed moderate CPU overhead due to swapping list contents. Java, by contrast, benefitted from explicit node re-linking, making Insertion Sort particularly efficient in its implementation. Despite these differences, the overall trends remained consistent: Bubble Sort was CPU-intensive, while Insertion Sort was more resource-friendly.

Beyond algorithm correctness, this assignment emphasized the importance of system-level performance monitoring. Tools such as htop, pidstat, iostat, and dstat provided insights into how algorithmic design choices translate into real-world resource usage. Although capturing precise

snapshots of resource utilization was challenging due to the continuously refreshing nature of these tools, the observed logs clearly validated theoretical predictions.

In conclusion, this project demonstrated not only how sorting algorithms can be implemented within a linked list–based memory database, but also how empirical measurements provide valuable evidence of algorithm efficiency. The exercise reinforced the importance of pairing theoretical analysis with practical experimentation, bridging the gap between algorithm design, programming language implementation, and operating system performance.

VII. Acknowledgment

I would like to thank **Professor Dai** for guidance and clear instructions throughout this assignment. The project helped me connect concepts of linked lists and recursion with practical implementation in Python and Java. I also acknowledge the use of the *student-data.csv* dataset from **Kaggle**, which provided a meaningful basis for testing the memory database. The availability of Linux tools such as htop, pidstat, iostat, and dstat was invaluable in analyzing CPU and disk performance. Finally, I am grateful for the feedback and support received during the development and testing process.

VIII. References

- Kelly Gakii, Student Data Dataset, Kaggle. Available at: https://www.kaggle.com/datasets/kellygakii/student-data-csv

- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.
- htop – Interactive Process Viewer. Linux manual pages. Available at: https://man7.org/linux/man-pages/man1/htop.1.html
- pidstat – Linux Process Statistics. Linux manual pages. Available at: https://man7.org/linux/man-pages/man1/pidstat.1.html
- iostat – Report CPU and I/O Statistics. Linux manual pages. Available at: https://man7.org/linux/man-pages/man1/iostat.1.html
- dstat – Versatile Resource Statistics Tool. Linux manual pages. Available at: https://linux.die.net/man/1/dstat
- OnlineGDB – Online Compiler and Debugger. Available at: https://www.onlinegdb.com
- GitHub – Code Hosting Platform. Available at: https://github.com

Appendix

i. Appendix A: Python Code

```
import csv

class Node:
    def __init__(self, data=None):
        self.data = data
        self.prev = None
        self.next = None
```

```
class MemoryDB:
    def __init__(self):
        self.head = None

    def add_record(self, student_data):
        new_node = Node(student_data)
        if self.head is None:
            self.head = new_node
        else:
            cur = self.head
            while cur.next:
                cur = cur.next
            cur.next = new_node
            new_node.prev = cur

    def _reload_recursive(self, reader, i=0):
        try:
            row = next(reader)
            if row:
                self.add_record([str(i)] + row)
            self._reload_recursive(reader, i+1)
        except StopIteration:
            return

    # (Option 1) Reload entire list (recursive)
    def reload_from_csv(self, csv_path):
        with open(csv_path, 'r', encoding='utf-8', newline='') as f:
            reader = csv.reader(f)
            header = next(reader)
            self.head = None
            self._reload_recursive(reader)
        return ["RowID"] + header

    # Export recursively
    def export_to_csv_recursive(self, writer, node):
        if node is None:
            return
        writer.writerow(node.data)
        self.export_to_csv_recursive(writer, node.next)
```

```
    # Bubble Sort on linked list
    def bubble_sort(self, key_index=1):
        if self.head is None:
            return
        swapped = True
        while swapped:
            swapped = False
            cur = self.head
            while cur.next:
                if cur.data[key_index] > cur.next.data[key_index]:
                    cur.data, cur.next.data = cur.next.data, cur.data
                    swapped = True
                cur = cur.next

    # Insertion Sort on linked list
    def insertion_sort(self, key_index=1):
        if self.head is None or self.head.next is None:
            return

        sorted_head = self.head
        cur = self.head.next
        sorted_head.next = None

        while cur:
            next_node = cur.next
            if cur.data[key_index] < sorted_head.data[key_index]:
                # insert at beginning
                cur.next = sorted_head
                sorted_head.prev = cur
                cur.prev = None
                sorted_head = cur
            else:
                search = sorted_head
                while search.next and search.next.data[key_index] <
cur.data[key_index]:
                    search = search.next
                cur.next = search.next
                if search.next:
                    search.next.prev = cur
```

```
                search.next = cur
                cur.prev = search
            cur = next_node

        self.head = sorted_head

    # Pretty print
    def display(self, max_rows=10):
        if not self.head:
            print("<empty>")
            return
        cur = self.head
        print("RowID | Data (first few columns)")
        print("-" * 70)
        count = 0
        while cur and count < max_rows:
            print(", ".join(cur.data[:6]), "...")
            cur = cur.next
            count += 1
        if cur:
            print("... (more rows)")


# ---------- Menu-driven program ----------
def main():
    csv_file = "student-data.csv"
    out_file = "updated_student-data.csv"

    db = MemoryDB()
    header = []

    while True:
        print("\n===== Memory Database Menu =====")
        print("1. Load from CSV (recursive)")
        print("2. Bubble Sort")
        print("3. Insertion Sort")
        print("4. Export DB to CSV (recursive)")
        print("5. Exit")

        try:
```

```
            choice = input("Enter choice: ").strip()
        except KeyboardInterrupt:
            print("\nExiting program...")
            exit(0)

        if choice == "1":
            header = db.reload_from_csv(csv_file)
            print("[OK] Loaded from file.")
            print("Columns:", header)
            db.display()

        elif choice in ["2", "3"]:
            if not header:
                print("[ERR] Load first using option 1.")
            else:
                print("Available columns for sorting:")
                for i, col in enumerate(header):
                    print(f"{i}. {col}")
                try:
                    key_index = int(input("Enter column index to sort by:
").strip())
                except ValueError:
                    print("[ERR] Invalid input.")
                    continue

                if choice == "2":
                    db.bubble_sort(key_index=key_index)
                    print(f"[OK] Bubble Sorted by {header[key_index]}")
                else:
                    db.insertion_sort(key_index=key_index)
                    print(f"[OK] Insertion Sorted by {header[key_index]}")
                db.display()

        elif choice == "4":
            if not header:
                print("[ERR] Nothing loaded yet.")
            else:
                with open(out_file, "w", newline='', encoding='utf-8') as
f:
                    writer = csv.writer(f)
```

```
                    writer.writerow(header)
                    db.export_to_csv_recursive(writer, db.head)
                print(f"[OK] Exported to {out_file}")

        elif choice == "5":
            print("Goodbye!")
            break

        else:
            print("Invalid choice, try again.")

if __name__ == "__main__":
    main()
```

ii. Appendix B: Java Code

```
import java.io.*;
import java.util.*;

class Node {
    List<String> data;
    Node prev;
    Node next;

    Node(List<String> data) {
        this.data = data;
        this.prev = null;
        this.next = null;
    }
}

class MemoryDB {
    private Node head = null;

    // Add record at the end
    public void addRecord(List<String> studentData) {
        Node newNode = new Node(studentData);
        if (head == null) {
```

```
            head = newNode;
        } else {
            Node cur = head;
            while (cur.next != null) {
                cur = cur.next;
            }
            cur.next = newNode;
            newNode.prev = cur;
        }
    }

    // Recursive loader helper
    private void reloadRecursive(Iterator<String[]> iter, int i) {
        if (!iter.hasNext()) return;
        String[] row = iter.next();
        if (row.length > 0) {
            List<String> newRow = new ArrayList<>();
            newRow.add(String.valueOf(i)); // RowID
            newRow.addAll(Arrays.asList(row));
            addRecord(newRow);
        }
        reloadRecursive(iter, i + 1);
    }

    // Reload from CSV
    public List<String> reloadFromCSV(String csvPath) throws IOException {
        try (BufferedReader br = new BufferedReader(new
FileReader(csvPath))) {
            String headerLine = br.readLine();
            if (headerLine == null) return Collections.emptyList();
            String[] header = headerLine.split(",");

            // reset linked list
            head = null;

            // collect all rows
            List<String[]> rows = new ArrayList<>();
            String line;
            while ((line = br.readLine()) != null) {
                rows.add(line.split(","));
```

```
            }
            reloadRecursive(rows.iterator(), 0);

            List<String> headerList = new ArrayList<>();
            headerList.add("RowID");
            headerList.addAll(Arrays.asList(header));
            return headerList;
        }
    }

    // Recursive export
    public void exportToCSVRecursive(PrintWriter pw, Node node) {
        if (node == null) return;
        pw.println(String.join(",", node.data));
        exportToCSVRecursive(pw, node.next);
    }

    // Bubble Sort
    public void bubbleSort(int keyIndex) {
        if (head == null) return;
        boolean swapped;
        do {
            swapped = false;
            Node cur = head;
            while (cur.next != null) {
                if
(cur.data.get(keyIndex).compareTo(cur.next.data.get(keyIndex)) > 0) {
                    List<String> temp = cur.data;
                    cur.data = cur.next.data;
                    cur.next.data = temp;
                    swapped = true;
                }
                cur = cur.next;
            }
        } while (swapped);
    }

    // Insertion Sort
    public void insertionSort(int keyIndex) {
        if (head == null || head.next == null) return;
```

```
        Node sorted = head;
        Node cur = head.next;
        sorted.next = null;

        while (cur != null) {
            Node nextNode = cur.next;
            if
(cur.data.get(keyIndex).compareTo(sorted.data.get(keyIndex)) < 0) {
                // insert at beginning
                cur.next = sorted;
                sorted.prev = cur;
                cur.prev = null;
                sorted = cur;
            } else {
                Node search = sorted;
                while (search.next != null &&

search.next.data.get(keyIndex).compareTo(cur.data.get(keyIndex)) < 0) {
                    search = search.next;
                }
                cur.next = search.next;
                if (search.next != null) {
                    search.next.prev = cur;
                }
                search.next = cur;
                cur.prev = search;
            }
            cur = nextNode;
        }
        head = sorted;
    }

    // Display (first few columns for readability)
    public void display(int maxRows) {
        if (head == null) {
            System.out.println("<empty>");
            return;
        }
        Node cur = head;
```

```
        int count = 0;
        while (cur != null && count < maxRows) {
            System.out.println(String.join(", ", cur.data));
            cur = cur.next;
            count++;
        }
        if (cur != null) {
            System.out.println("... (more rows)");
        }
    }

    public Node getHead() {
        return head;
    }
}

public class MemoryDBApp {
    public static void main(String[] args) {
        String csvFile = "student-data.csv";
        String outFile = "updated_student-data.csv";

        MemoryDB db = new MemoryDB();
        List<String> header = new ArrayList<>();

        Scanner sc = new Scanner(System.in);

        while (true) {
            System.out.println("\n===== Memory Database Menu =====");
            System.out.println("1. Load from CSV (recursive)");
            System.out.println("2. Bubble Sort");
            System.out.println("3. Insertion Sort");
            System.out.println("4. Export DB to CSV (recursive)");
            System.out.println("5. Exit");
            System.out.print("Enter choice: ");
            String choice = sc.nextLine().trim();

            try {
                switch (choice) {
                    case "1":
                        header = db.reloadFromCSV(csvFile);
```

```
                        System.out.println("[OK] Loaded from file.");
                        System.out.println("Columns: " + header);
                        db.display(10);
                        break;

                    case "2":
                    case "3":
                        if (header.isEmpty()) {
                            System.out.println("[ERR] Load first using
option 1.");
                            break;
                        }
                        // show available columns
                        System.out.println("Available columns for
sorting:");
                        for (int i = 0; i < header.size(); i++) {
                            System.out.println(i + ". " + header.get(i));
                        }
                        System.out.print("Enter column index: ");
                        int keyIndex =
Integer.parseInt(sc.nextLine().trim());

                        if (choice.equals("2")) {
                            db.bubbleSort(keyIndex);
                            System.out.println("[OK] Bubble Sorted by " +
header.get(keyIndex));
                        } else {
                            db.insertionSort(keyIndex);
                            System.out.println("[OK] Insertion Sorted by "
+ header.get(keyIndex));
                        }
                        db.display(10);
                        break;

                    case "4":
                        if (header.isEmpty()) {
                            System.out.println("[ERR] Nothing loaded
yet.");
                        } else {
```

```
                            try (PrintWriter pw = new PrintWriter(new
FileWriter(outFile))) {
                                pw.println(String.join(",", header));
                                db.exportToCSVRecursive(pw, db.getHead());
                            }
                            System.out.println("[OK] Exported to " +
outFile);
                        }
                        break;

                    case "5":
                        System.out.println("Goodbye!");
                        return;

                    default:
                        System.out.println("Invalid choice, try again.");
                }
            } catch (IOException e) {
                System.out.println("[ERR] " + e.getMessage());
            }
        }
    }
}
```